# Quiz 2 Study Guide — Lectures 4–7

CSCE 775 — Spring 2026

J.C. Vaught

**Quiz format:** 1 math problem (50%) + conceptual questions (50%). Strategy: nail the conceptual questions, get partial credit on the math.

## Part I

# Lecture 4: Model-Free RL

*Covers Sutton & Barto Ch. 5.1–5.7, 6.1–6.5, 7.1–7.3*

## 1 Big Picture

**Model-free RL** means learning from *experience* without knowing the transition function $p(s', r \mid s, a)$. Two tasks:

- **Prediction (policy evaluation):** estimate $v_\pi$ or $q_\pi$ for a given $\pi$
- **Control:** find the optimal policy $\pi_*$

Three families of methods, each with a different *target* for the update:

| Method | Target | Bias | Variance |
|---|---|---|---|
| Monte Carlo | $G_t$ (full return) | None | High |
| TD(0) | $R_{t+1} + \gamma V(S_{t+1})$ | Yes (bootstrap) | Low |
| $n$-step TD | $R_{t+1} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$ | Medium | Medium |

## 2 Monte Carlo Methods (Ch. 5)

### 2.1 MC Prediction

Estimate $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ by averaging *sampled returns*.

**MC Incremental Update:** $V(S_t) \leftarrow V(S_t) + \alpha\big(G_t - V(S_t)\big)$

- **First-visit MC:** average $G_t$ only the *first* time $s$ appears in an episode.
- **Every-visit MC:** average $G_t$ every time $s$ appears.
- Must wait until the episode ends to compute $G_t$. Does *not* work for continuing (non-episodic) tasks.

**Intuition:** MC is like grading a restaurant after eating the entire meal. You wait until the end, then assign a score to the whole experience. Accurate (unbiased), but your one meal might have been unusually good or bad (high variance).

**Example:** A robot plays 3 episodes starting from state $A$. It gets total returns $G = -5, -3, -7$. After first-visit MC: $V(A) = (-5 + -3 + -7)/3 = -5$. Incrementally: start $V(A) = 0$, after ep. 1: $V(A) = 0 + 1(-5 - 0) = -5$. After ep. 2: $V(A) = -5 + \frac{1}{2}(-3 - (-5)) = -4$. After ep. 3: $V(A) = -4 + \frac{1}{3}(-7 - (-4)) = -5$.

### 2.2 MC Control (On-Policy)

Use $Q(s, a)$ instead of $V(s)$ because we cannot derive a policy from $V$ without a model.

**Policy from Q:** $\pi(s) = \arg\max_a Q(s, a)$

**$\varepsilon$-greedy exploration:**

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(s)| & \text{if } a = \arg\max_{a'} Q(s, a') \\ \varepsilon/|\mathcal{A}(s)| & \text{otherwise} \end{cases}$$

The on-policy MC control algorithm: generate episode $\rightarrow$ compute returns backward $\rightarrow$ update $Q(S_t, A_t)$ $\rightarrow$ improve policy to $\varepsilon$-greedy w.r.t. new $Q$.

**Intuition:** $\varepsilon$-greedy is like a food critic who *usually* goes to their favorite restaurant (exploit, prob $1 - \varepsilon$) but *sometimes* tries a random place (explore, prob $\varepsilon$). Without exploration, you might never discover the best option.

**Example:** With $\varepsilon = 0.1$ and 4 actions, the greedy action gets probability $0.9 + 0.1/4 = 0.925$. Each non-greedy action gets $0.1/4 = 0.025$. Sum: $0.925 + 3(0.025) = 1$.

### 2.3 On-Policy vs Off-Policy

- **Behavior policy $b$:** the policy used to collect data.
- **Target policy $\pi$:** the policy we want to evaluate/improve.
- **On-policy:** $b = \pi$.
- **Off-policy:** $b \neq \pi$. Can reuse data from other policies. Requires importance sampling.

## 3 Temporal-Difference Learning (Ch. 6)

### 3.1 TD(0) Prediction

**Key idea:** update toward a *bootstrapped* target instead of waiting for $G_t$.

**TD(0) Update:** $V(S_t) \leftarrow V(S_t) + \alpha\big[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t)\big]$

The quantity $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is the **TD error**.

**Intuition:** TD is like updating your restaurant rating after *each course*, not waiting for the full meal. After the appetizer, you adjust your expectation based on what you just experienced + your guess about the rest. Faster feedback, but your guess about dessert might be wrong (biased).

**Example:** States $A \to B$ with $r = 0$, $\gamma = 1$, $\alpha = 0.1$. Initially $V(A) = 0$, $V(B) = 0.75$.
TD update: $V(A) \leftarrow 0 + 0.1[0 + 1(0.75) - 0] = 0.075$.
TD immediately propagates $B$'s value back to $A$ after one step. MC would need to wait for the episode to finish.

**MC vs TD:**

- MC: unbiased, high variance, needs full episodes.
- TD: biased (initially), low variance, learns online step-by-step.
- TD bootstraps from its own estimate ⇒ can propagate value information faster.

### 3.2 SARSA — On-Policy TD Control

**SARSA Update:** $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big]$

Name comes from the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$.

Both behavior and target policy are $\varepsilon$-greedy. Choose $A_{t+1}$ from $S_{t+1}$ using $\varepsilon$-greedy *before* the update.

**Intuition:** SARSA asks: "given what I *actually did* next (including random exploration), how good was my action?" It's cautious—if exploration might lead off a cliff, SARSA learns to stay away.

### 3.3 Q-Learning — Off-Policy TD Control

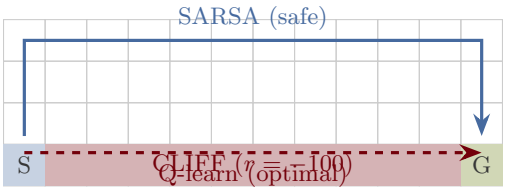
**Q-Learning Update:** $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\big]$

**Key difference:** uses $\max_a Q(S_{t+1}, a)$ (greedy) instead of $Q(S_{t+1}, A_{t+1})$ ($\varepsilon$-greedy).

- Behavior policy: $\varepsilon$-greedy (explores).
- Target policy: greedy (deterministic).
- Off-policy because behavior $\neq$ target.

**Intuition:** Q-learning asks: "what's the best I *could* do next, assuming I play perfectly from now on?" It's optimistic—it finds the optimal path even if exploration is risky.

**Cliff Walking Example:**




- SARSA learns a *safer* path (avoids cliff) because it accounts for its own $\varepsilon$-greedy exploration.
- Q-learning learns the *optimal* path (close to cliff) because the target policy is greedy.

## 4 $n$-Step Methods (Ch. 7)

**$n$-step return:** $G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$

- $n = 1$: TD(0).
- $n = \infty$: Monte Carlo.
- Typical sweet spot: $n \approx 5$.

**TD($\lambda$)** averages over *all* $n$-step returns with exponentially decaying weights $(1-\lambda)\lambda^{n-1}$. When $\lambda = 0$ you get TD(0); when $\lambda = 1$ you get MC.

**Intuition:** $n$-step methods are like reading $n$ chapters of a mystery novel before guessing the ending. $n = 1$ (TD): guess after the first chapter—fast but unreliable. $n = \infty$ (MC): read the whole book—accurate but slow. Best results come from reading a few chapters ($n \approx 5$) then making an informed guess.

**Example:** In a 4-step episode with rewards $r_1 = 1, r_2 = 0, r_3 = 1, r_4 = 2$ (terminal) and $\gamma = 1$:
**1-step return:** $G_{1:2} = 1 + V(S_2)$ (uses one real reward + estimate)
**2-step return:** $G_{1:3} = 1 + 0 + V(S_3)$ (two real rewards + estimate)
**4-step (MC):** $G_1 = 1 + 0 + 1 + 2 = 4$ (all real rewards, no estimate)

## 5 Unified View: DP vs TD

| | **Full Backup (DP)** | **Sample Backup (TD)** |
|---|---|---|
| Bellman Expectation for $v_\pi$ | Policy Evaluation | TD Learning |
| Bellman Expectation for $q_\pi$ | Q-Policy Iteration | SARSA |
| Bellman Optimality for $q_*$ | Q-Value Iteration | Q-Learning |

**Key difference:** DP uses the full model (all transitions); TD samples one transition.

Part II

# Lecture 5: Deep Learning Fundamentals

## 6 Linear Models

### 6.1 Linear Regression

$f(\mathbf{x}, \mathbf{w}, b) = \mathbf{w}^\top \mathbf{x} + b$ **Loss:** $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - f(\mathbf{x}_i, \boldsymbol{\theta}))^2$

**Analytical solution:** $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

**Gradient descent update:** $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$

**Intuition:** Linear regression fits a line (or hyperplane) that minimizes the total squared distance to the data points. Gradient descent nudges the line toward data points it currently misses most.

**Example:** Predicting house price from square footage: $f(x) = w \cdot x + b$. If $w = 200, b = 50000$, a 1500 sq ft house costs \$350,000. The loss measures how far off our predictions are from actual prices.

### 6.2 Logistic Regression (Binary Classification)

**Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$ **Derivative:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$. Decision boundary where $\mathbf{w}^\top \mathbf{x} = 0$.

**Loss:** negative log-likelihood (cross-entropy):

$$\mathcal{L} = -\sum_i \left[ y_i \log \sigma(\mathbf{w}^\top \mathbf{x}_i) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^\top \mathbf{x}_i)) \right]$$

**Intuition:** Sigmoid squashes any real number into $(0, 1)$—like a dimmer switch that smoothly goes from "off" to "on." For $z \gg 0$, output $\approx 1$; for $z \ll 0$, output $\approx 0$. The cross-entropy loss punishes confident wrong answers exponentially.

**Example:** Email spam classification: $\mathbf{w}^\top \mathbf{x} = 3.5 \Rightarrow \sigma(3.5) = 0.97 \Rightarrow 97\%$ chance of spam. If actually spam ($y = 1$): loss $= -\log(0.97) = 0.03$ (small). If not spam ($y = 0$): loss $= -\log(0.03) = 3.5$ (large penalty for being confidently wrong).

### 6.3 Softmax (Multi-class)

$$P(y = i \mid \mathbf{x}) = \frac{e^{\mathbf{w}_i^\top \mathbf{x}}}{\sum_{j=1}^{C} e^{\mathbf{w}_j^\top \mathbf{x}}}$$

# 7 Neural Networks

## 7.1 Architecture

A 2-layer network: $f(\mathbf{x}) = \mathbf{W}^{(2)}\sigma(\mathbf{W}^{(1)}\mathbf{x})$

**Activation functions must be non-linear**, otherwise the network collapses to a linear model regardless of depth.

**Intuition:** A neural network without non-linear activations is just matrix multiplication stacked: $\mathbf{W}_2(\mathbf{W}_1\mathbf{x}) = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} = \mathbf{W}'\mathbf{x}$. No matter how many layers, it is still linear. Non-linear activations let the network "bend" the decision boundary into curves, spirals, and arbitrary shapes.

**Example:** XOR problem: inputs $(0,0) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$, $(1,1) \to 0$. No single line separates the classes. A 2-layer network with ReLU can solve this; a linear model cannot.

## 7.2 Common Activation Functions

| Name | Formula | Notes |
|---|---|---|
| Sigmoid | $\sigma(x) = 1/(1+e^{-x})$ | Output in $(0,1)$; vanishing gradient |
| Tanh | $\tanh(x)$ | Output in $(-1,1)$ |
| ReLU | $\max(0,x)$ | Most common; dead neurons possible |
| Leaky ReLU | $\max(0.1x,x)$ | Fixes dead neuron problem |

## 7.3 Universal Function Approximation

With enough hidden units, a neural network can approximate *any* continuous function to arbitrary precision. Does *not* guarantee convergence of training.

## 7.4 Backpropagation

Chain rule applied layer by layer. For output layer weights:

$$\frac{\partial E}{\partial W_{ij}^{(2)}} = -(y_i - \hat{y}_i)\, h_j$$

For hidden layer weights, errors propagate backward through the chain rule.

**Intuition:** Backpropagation is the chain rule applied systematically. Think of a pipeline: raw material → factory 1 → factory 2 → product. If the product is bad, you trace blame backward: How much did factory 2 mess up? Given factory 2's error, how much did factory 1 contribute? Each factory adjusts proportionally to its blame.

## 7.5 Hyperparameters vs Parameters

- **Parameters:** learned from data (weights, biases).
- **Hyperparameters:** set before training (learning rate, # layers, # neurons, $\varepsilon$, etc.).

# 8 Overfitting and Regularization

| Technique | How it works |
|---|---|
| Train/Val/Test split | Monitor val loss; never train on test |
| Weight regularization | Add $\lambda \sum W_{ij}^2$ to loss |
| Dropout | Randomly zero out neurons during training |
| Data augmentation | Rotations, flips, noise to increase data |
| Early stopping | Stop when validation error starts rising |

**Intuition:** Overfitting is like memorizing the textbook word-for-word instead of understanding the concepts. You ace the homework (training set) but fail the exam (test set). Regularization techniques force the model to learn *general patterns* instead of memorizing noise.

**Example:** A polynomial of degree 100 can pass through 100 training points perfectly (training loss = 0) but wildly oscillates between them. A degree-2 polynomial has higher training loss but generalizes much better to new data.

# 9 Optimization

Loss surface is non-convex: local minima, saddle points.

| Optimizer | Update Rule |
|---|---|
| SGD | $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla E$ |
| SGD + Momentum | $\mathbf{v} \leftarrow \mu \mathbf{v} + \alpha \nabla E; \quad \mathbf{w} \leftarrow \mathbf{w} - \mathbf{v}$ |
| ADAM | Adaptive lr per parameter using mean & variance of gradients |

**Stochastic GD:** compute gradients on random mini-batches instead of full dataset.

## 9.1 Residual Networks (ResNets)

Skip connections: output $= F(\mathbf{x}) + \mathbf{x}$. Smooths the loss surface; enables training very deep networks (100+ layers).

## 9.2 Batch Normalization

Normalize activations to mean 0, std 1 within each mini-batch. Stabilizes training, allows larger learning rates.

**Intuition:** **SGD with momentum** is like a ball rolling downhill—it accumulates speed and can roll past small bumps (local minima). **ADAM** is like a ball that also adjusts its speed independently in each direction, moving faster along flat dimensions and slower along steep ones.
**ResNets:** In deep networks, gradients can vanish layer by layer. Skip connections provide a "highway" for gradients to flow directly backward, so even layer 100 gets a strong learning signal.
**Batch norm:** Without it, each layer's input distribution shifts as earlier layers update (internal covariate shift). Batch norm re-centers activations each step, so layers don't have to chase a moving target.

**Example:** Training a 20-layer network: without skip connections, the gradient at layer 1 is $\approx 0$ (vanishing gradient). With skip connections, the gradient flows through the shortcut: $\frac{\partial}{\partial \mathbf{x}}[F(\mathbf{x}) + \mathbf{x}] = F'(\mathbf{x}) + \mathbf{I}$. The identity $\mathbf{I}$ ensures the gradient is at least 1.

Part III

# Lecture 6: PyTorch

## 10 Key Concepts (Likely Conceptual Questions)

1. **Automatic differentiation:** define forward pass; PyTorch computes backward pass (gradients) automatically via computation graph.
2. **Tensor:** multi-dimensional array (like numpy ndarray), but tracks gradients.
3. **Computation graph:** directed graph of operations. Forward pass builds it; `loss.backward()` traverses it backward.
4. **`.detach()`:** removes a tensor from the computation graph (stops gradient flow).
5. **Data type:** PyTorch expects `float32`. Convert before or after creating tensors.
6. **`loss.item()`:** extract scalar value without keeping the computation graph (prevents memory leak).
7. **`nnet.train()` vs `nnet.eval()`:** training mode enables dropout and batch norm updates; eval mode disables them.
8. **Saving/loading:** `torch.save(nnet.state_dict(), path)` saves only parameters, not architecture.

## 11 Training Loop Pattern

1. `optimizer.zero_grad()` — clear old gradients
2. Forward pass: `output = nnet(input)`
3. Compute loss: `loss = criterion(output, target)`
4. Backward pass: `loss.backward()`
5. Update weights: `optimizer.step()`

## 12 ResNet in PyTorch

Residual block: two linear layers + skip connection. Requires input dim = hidden dim (or use a linear projection). Forward: $\mathbf{x} \leftarrow \text{ReLU}(F(\mathbf{x}) + \mathbf{x})$.

## 13 Gradient Checking

Finite differences: $\frac{\partial f}{\partial \theta_i} \approx \frac{f(\theta_i + h) - f(\theta_i)}{h}$ for small $h$ (e.g., $10^{-5}$). Verify each layer independently, then compose.

**Intuition:** PyTorch's autograd is like having an accountant who watches every arithmetic operation you do (forward pass) and records it in a ledger (computation graph). When you call `.backward()`, the accountant traces the ledger in reverse to compute exactly how much each input contributed to the final result. `.detach()` tells the accountant to stop recording—useful for targets that should be treated as constants (like in DQN).

**Example:** The 5-step PyTorch training loop maps to calculus:
1. `zero_grad()` $\rightarrow$ erase old $\nabla_\theta$ 2. Forward $\rightarrow$ compute $\hat{y}$ 3. Loss $\rightarrow$ compute $\mathcal{L}$ 4. `backward()` $\rightarrow$ compute $\nabla_\theta \mathcal{L}$ 5. `step()` $\rightarrow \theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$
Forgetting `zero_grad()` accumulates gradients across batches (gradients from batch 2 are added to batch 1's gradients).

Part IV

# Lecture 7: DAVI and Deep Q-Networks

## 14 Deep Approximate Value Iteration (DAVI)

When the state space is too large for a table, approximate $v_*(s)$ with a neural network $v_\theta(s)$.

**DAVI Target:** $y = \begin{cases} \max_a \big(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\theta(s')\big) & \text{if } s \text{ non-terminal} \\ 0 & \text{if } s \text{ terminal} \end{cases}$

**Loss:** $L(\theta) = \frac{1}{2}(y - v_\theta(s))^2$ **Gradient:** $\nabla_\theta L = (y - v_\theta(s))\nabla_\theta v_\theta(s)$

**Do NOT differentiate $y$ with respect to $\theta$** even though $y$ depends on $\theta$. Treat $y$ as a fixed target.

**Intuition:** DAVI is value iteration (from DP) but with a neural network instead of a table. Normally, value iteration updates every state at once using the model. DAVI does the same thing but trains a neural network to approximate the value function. The target $y$ is the "what the value *should* be" (one-step lookahead), and we train the network to match it.

**Example:** Grid world with 1 million states—too many for a table. DAVI: sample a batch of states, compute targets $y = \max_a [r + \gamma \sum_{s'} p(s'|s,a) v_{\theta^-}(s')]$ using the frozen target network, then do one gradient step to make $v_\theta(s)$ closer to $y$. Repeat.

### 14.1 Target Network

Problem: target $y$ changes every iteration (non-stationary).

Solution: maintain a separate **target network** $v_{\theta^-}$ whose parameters are periodically copied from $\theta$. Use $\theta^-$ to compute $y$, train $\theta$.

**Intuition:** Without a target network, you're chasing a moving target: every time you update $\theta$, the target $y$ changes too (because $y$ depends on $v_\theta$). It's like trying to hit a bullseye that moves every time you throw. The target network freezes the bullseye for $C$ steps, letting you make progress.

## 15 Deep Q-Networks (DQN)

Approximate $q_*(s,a)$ with a neural network. Input: state $s$. Output: $Q$-values for all actions (one forward pass).

**DQN Target:** $y = \begin{cases} r + \gamma \max_{a'} q_{\theta^-}(s', a') & \text{non-terminal} \\ r & \text{terminal} \end{cases}$

**DQN Loss:** $L(\theta) = \frac{1}{2}(y - q_\theta(s,a))^2$

### 15.1 Three Key DQN Innovations

1. **Target network** ($\theta^-$ updated periodically) — stabilizes training by making targets less non-stationary.
2. **Experience replay buffer** — store $(s, a, r, s')$ tuples; sample random mini-batches to decorrelate training data and prevent catastrophic forgetting.
3. **Huber loss** — clips gradients for large TD errors, preventing instability from large rewards.

### 15.2 DQN Ablation (know this!)

Both replay buffer AND target network are essential. Removing either one drastically hurts performance. DQN also vastly outperforms linear function approximators.

**Intuition:** DQN = Q-learning + neural network + two tricks.
**Experience replay:** Instead of learning from experiences in order (which are correlated—consecutive frames in a game look alike), store them in a buffer and sample random mini-batches. This breaks correlations, just like shuffling a deck of flashcards helps you learn better than going in order.
**Target network + replay** are *both* essential. The ablation study shows removing either one causes catastrophic failure. They solve different problems: replay breaks correlation, target network prevents oscillation.

**Example:** Atari Breakout: state = raw pixels (84×84×4 frames). Action = left/right/fire. DQN outputs $Q(s, a)$ for all 3 actions in one forward pass. The replay buffer stores millions of $(s, a, r, s')$ transitions. Every 10,000 steps, copy $\theta \to \theta^-$.

## 16 DQN Improvements

| Method | Key Idea |
|---|---|
| **Double Q-Learning** | Two Q-networks: one selects the action, the other evaluates it. Fixes *maximization bias*. $y = r + \gamma q_{\theta_2}(s', \arg\max_{a'} q_{\theta_1}(s', a'))$ |
| **Dueling DQN** | Split network into value stream $v_\theta(s)$ and advantage stream $A_\theta(s, a)$. $q(s, a) = v_\theta(s) + A_\theta(s, a)$. Advantage: $A_\pi(s, a) = q_\pi(s, a) - v_\pi(s)$. |
| **Prioritized Replay** | Sample transitions with large TD error more frequently. |
| **Rainbow** | Combines DQN + Double + Dueling + Prioritized + others. Best overall. |
| **Agent57** | First to beat humans on all 57 Atari games. Uses intrinsic curiosity + adaptive discount. |

**Intuition:** **Double Q-learning** fixes maximization bias: if you ask the same person to both nominate and evaluate candidates, they'll be overconfident. Use one network to *pick* the best action, a different network to *score* it.
**Dueling DQN** separates "how good is this state?" ($v(s)$) from "how much better is this action than average?" ($A(s,a)$). Many states have similar values regardless of action—the dueling architecture learns this shared value efficiently.

## Part V

# Likely Derivation Problems

The math problem is 50% and is a pure symbolic derivation (no numbers). Here are the most likely candidates with full step-by-step derivations.

## 17 Derivation 1: MC Incremental Update from Running Average

*"Show that the running-average MC update can be written in incremental form."*

Start from the sample-mean definition after $n$ visits to state $s$:

$$V_n = \frac{1}{n}\sum_{k=1}^{n} G_k$$

Split off the $n$-th return:

$$V_n = \frac{1}{n}\Big(G_n + \sum_{k=1}^{n-1} G_k\Big) = \frac{1}{n}\Big(G_n + (n-1)\,V_{n-1}\Big) \tag{1}$$

$$= \frac{1}{n}\,G_n + \frac{n-1}{n}\,V_{n-1} = \frac{1}{n}\,G_n + V_{n-1} - \frac{1}{n}\,V_{n-1} \tag{2}$$

$$= V_{n-1} + \frac{1}{n}\big(G_n - V_{n-1}\big) \tag{3}$$

$V_n = V_{n-1} + \alpha_n\big(G_n - V_{n-1}\big)$ where $\alpha_n = 1/n$
Generalizing to a constant step size: $V(S_t) \leftarrow V(S_t) + \alpha\big(G_t - V(S_t)\big)$

**Intuition:** This derivation shows that "take the average of all returns so far" is equivalent to "nudge the old estimate toward the new return." The key algebraic trick is splitting off the last term from the sum and rewriting $\frac{n-1}{n} = 1 - \frac{1}{n}$.

## 18 Derivation 2: $\varepsilon$-Greedy Policy Improvement

*"Prove that for any $\varepsilon$-greedy policy $\pi$, the $\varepsilon$-greedy policy $\pi'$ w.r.t. $q_\pi$ is an improvement: $v_{\pi'}(s) \geq v_\pi(s)$."*

Start with the expected value under $\pi'$:

$$q_\pi(s, \pi'(s)) = \sum_a \pi'(a \mid s)\, q_\pi(s,a) = \frac{\varepsilon}{|\mathcal{A}|}\sum_a q_\pi(s,a) + (1-\varepsilon)\max_a q_\pi(s,a) \tag{4}$$

Since $\max_a q_\pi(s,a) \geq \sum_a w_a\, q_\pi(s,a)$ for any convex weights $w_a$ summing to 1, substitute $w_a = \frac{\pi(a|s) - \varepsilon/|\mathcal{A}|}{1-\varepsilon}$:

$$\geq \frac{\varepsilon}{|\mathcal{A}|}\sum_a q_\pi(s,a) + (1-\varepsilon)\sum_a \frac{\pi(a \mid s) - \varepsilon/|\mathcal{A}|}{1-\varepsilon}\, q_\pi(s,a) \tag{5}$$

$$= \frac{\varepsilon}{|\mathcal{A}|}\sum_a q_\pi(s,a) + \sum_a \big(\pi(a \mid s) - \varepsilon/|\mathcal{A}|\big)\, q_\pi(s,a) \tag{6}$$

$$= \sum_a \pi(a \mid s)\, q_\pi(s,a) = v_\pi(s) \tag{7}$$

$q_\pi(s, \pi'(s)) \geq v_\pi(s)$ for all $s$. By the **policy improvement theorem**, this implies $v_{\pi'}(s) \geq v_\pi(s)$ for all $s$. $\square$

**Intuition:** The trick is showing the $\varepsilon$-greedy action value is at least as good as the old policy. The max in $\varepsilon$-greedy is $\geq$ any weighted average (including the old policy's weights), because max $\geq$ average.

## 19 Derivation 3: Sigmoid Derivative

*"Derive $\frac{d}{dz}\sigma(z)$ where $\sigma(z) = \frac{1}{1+e^{-z}}$."*

$$\sigma(z) = (1 + e^{-z})^{-1} \tag{8}$$

$$\frac{d\sigma}{dz} = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} \tag{9}$$

$$= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z) \cdot \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} \tag{10}$$

$$= \sigma(z)\big(1 - \sigma(z)\big) \tag{11}$$

$$\frac{d\sigma}{dz} = \sigma(z)\big(1 - \sigma(z)\big)$$

**Intuition:** The sigmoid derivative is maximized at $z = 0$ (where $\sigma = 0.5$, so $\sigma' = 0.25$) and vanishes for $|z| \gg 0$. This is why deep sigmoid networks suffer from vanishing gradients: each layer multiplies by at most 0.25.

## 20 Derivation 4: Gradient of Logistic Regression Loss

*"Derive the gradient of the cross-entropy loss for logistic regression."*

The log-likelihood for one sample:

$$\ell = y \log \sigma(\mathbf{w}^\top \mathbf{x}) + (1 - y) \log(1 - \sigma(\mathbf{w}^\top \mathbf{x}))$$

Using $\sigma' = \sigma(1 - \sigma)$ and abbreviating $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x})$:

$$\frac{\partial \ell}{\partial \mathbf{w}} = y \cdot \frac{\hat{y}(1 - \hat{y})}{\hat{y}} \mathbf{x} + (1 - y) \cdot \frac{-\hat{y}(1 - \hat{y})}{1 - \hat{y}} \mathbf{x} \tag{12}$$

$$= y(1 - \hat{y})\,\mathbf{x} - (1 - y)\hat{y}\,\mathbf{x} = (y - \hat{y})\,\mathbf{x} \tag{13}$$

The loss is *negative* log-likelihood, so the gradient of the loss over $N$ samples:

$$\nabla_{\mathbf{w}} \mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)\,\mathbf{x}_i = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)\,\mathbf{x}_i \quad \text{where } \hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$$

**Intuition:** The gradient has a beautiful form: $(\hat{y} - y)\mathbf{x}$. The update is proportional to the error $(\hat{y} - y)$ in the direction of the input $(\mathbf{x})$. Large errors $\rightarrow$ large updates. The key step is using $\sigma' = \sigma(1 - \sigma)$ to cancel the denominators from the log derivatives.

## 21 Derivation 5: Backpropagation for a 2-Layer Network

*"Derive the weight gradients for a 2-layer network."*

Network: $\hat{\mathbf{y}} = \mathbf{W}^{(2)}\,\mathbf{h}$ where $\mathbf{h} = \sigma(\mathbf{W}^{(1)}\mathbf{x})$. Loss: $E = \frac{1}{2}\|\mathbf{y} - \hat{\mathbf{y}}\|^2 = \frac{1}{2}\sum_i (y_i - \hat{y}_i)^2$.

**Output layer weights** (apply chain rule once):

$$\hat{y}_i = \sum_j W_{ij}^{(2)}\, h_j \tag{14}$$

$$\frac{\partial E}{\partial W_{ij}^{(2)}} = \frac{\partial E}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial W_{ij}^{(2)}} = -(y_i - \hat{y}_i) \cdot h_j \tag{15}$$

**Hidden layer weights** (chain rule through two layers):

$$h_j = \sigma\Big(\sum_k W_{jk}^{(1)} x_k\Big) \tag{16}$$

$$\frac{\partial E}{\partial W_{jk}^{(1)}} = \sum_i \frac{\partial E}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial h_j} \cdot \frac{\partial h_j}{\partial W_{jk}^{(1)}} = \sum_i \big[-(y_i - \hat{y}_i)\big]\, W_{ij}^{(2)}\; \sigma'\Big(\sum_k W_{jk}^{(1)} x_k\Big)\; x_k \tag{17}$$

**Pattern:** error at output × weight connecting layers × activation derivative × input. Errors "back-propagate" through each layer via the chain rule.

**Intuition:** Output layer gradient is simple: error × hidden activation. Hidden layer gradient adds one more factor: you sum the errors from all output neurons that this hidden unit connects to, weighted by the connection strengths. This "error propagation" is what makes it *back*propagation.

## 22 Derivation 6: DAVI / Approximate Value Iteration Gradient

*"Derive the gradient of the approximate value iteration loss."*

Target (treated as constant): $y = \max_a\big(r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v_{\theta^-}(s')\big)$

Loss for a single state:

$$L(\theta) = \frac{1}{2}\big(y - v_\theta(s)\big)^2$$

Apply the chain rule (do *not* differentiate $y$ w.r.t. $\theta$):

$$\nabla_\theta L(\theta) = \frac{\partial}{\partial \theta}\,\frac{1}{2}\big(y - v_\theta(s)\big)^2 = \big(y - v_\theta(s)\big) \cdot \big(-\nabla_\theta v_\theta(s)\big) \tag{18}$$

$\nabla_\theta L(\theta) = -\big(y - v_\theta(s)\big)\,\nabla_\theta v_\theta(s)$
The gradient descent update is then: $\theta \leftarrow \theta - \alpha\,\nabla_\theta L = \theta + \alpha\big(y - v_\theta(s)\big)\,\nabla_\theta v_\theta(s)$
**Key:** $y$ depends on $\theta^-$ (frozen target network), NOT on $\theta$, so it is treated as a constant.

**Intuition:** This is a standard supervised learning gradient—MSE loss between prediction $v_\theta(s)$ and target $y$. The only subtlety is that $y$ is *not* differentiated even though it depends on network parameters (via $\theta^-$). In PyTorch, this means computing $y$ with `.detach()` or `torch.no_grad()`.

For $N$ states (batched):

$$L(\theta) = \frac{1}{2N}\sum_{i=1}^{N}(y_i - v_\theta(s_i))^2 \quad \Rightarrow \quad \nabla_\theta L = -\frac{1}{N}\sum_{i=1}^{N}(y_i - v_\theta(s_i))\,\nabla_\theta v_\theta(s_i)$$

## 23 Derivation 7: Linear Regression Gradient (Closed Form)

*"Derive the analytical solution for linear regression."*

Model: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$. Loss:

$$\mathcal{L}(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = (\mathbf{X}\mathbf{w} - \mathbf{y})^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$$

Expand:

$$\mathcal{L} = \mathbf{w}^\top\mathbf{X}^\top\mathbf{X}\mathbf{w} - 2\mathbf{w}^\top\mathbf{X}^\top\mathbf{y} + \mathbf{y}^\top\mathbf{y} \tag{19}$$

Take gradient and set to zero:

$$\nabla_{\mathbf{w}}\mathcal{L} = 2\mathbf{X}^\top\mathbf{X}\mathbf{w} - 2\mathbf{X}^\top\mathbf{y} = \mathbf{0} \tag{20}$$

$$\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$$

**Intuition:** This is one of the few ML problems with an exact solution. Expand the squared loss, take the derivative (using matrix calculus), set to zero, and solve. The key identity is $\nabla_{\mathbf{w}}(\mathbf{w}^\top\mathbf{A}\mathbf{w}) = 2\mathbf{A}\mathbf{w}$. This closed form exists only because MSE + linear model = quadratic, which has a unique minimum.

## Part VI

# Rapid-Fire Conceptual Review

1. **Why Q instead of V for model-free control?** Cannot derive $\pi$ from $V(s)$ without knowing $p(s'|s,a)$. With $Q(s,a)$, just take $\arg\max_a$.
2. **MC vs TD bias/variance?** MC: unbiased, high variance. TD: biased (bootstraps), low variance.
3. **Can MC handle continuing tasks?** No. Must wait for episode to end.
4. **SARSA vs Q-learning?** SARSA is on-policy (uses $Q(S', A')$); Q-learning is off-policy (uses $\max_a Q(S', a)$). In cliff walking, SARSA is safer, Q-learning is optimal.
5. **What is bootstrapping?** Using current estimates of value to update other estimates (TD does this; MC does not).
6. **What is the TD error?** $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$.
7. **Why use $\varepsilon$-greedy?** Ensures exploration. Pure greedy can get stuck.
8. **Why must activations be non-linear?** Otherwise, any depth of network collapses to a single linear transformation.
9. **What is ReLU?** $\max(0, x)$. Simple, efficient, avoids vanishing gradient (for $x > 0$).
10. **What is a residual connection?** output $= F(\mathbf{x}) + \mathbf{x}$. Enables training deep networks by smoothing the loss surface.
11. **What is batch normalization?** Normalize layer inputs to mean 0, std 1 per mini-batch. Stabilizes and accelerates training.
12. **Overfitting solutions?** Regularization, dropout, data augmentation, early stopping, simpler model.
13. **Why experience replay?** Decorrelates training samples. Prevents catastrophic forgetting. Improves data efficiency.
14. **Why target network?** Stabilizes training by keeping the target fixed for many updates, reducing non-stationarity.
15. **What is maximization bias?** Q-learning overestimates Q-values because it uses max for both selection and evaluation. Double Q-learning fixes this.
16. **What is the advantage function?** $A_\pi(s,a) = q_\pi(s,a) - v_\pi(s)$. How much better action $a$ is than average.
17. **Dueling DQN?** Separate streams for $v(s)$ and $A(s,a)$, combined as $q(s,a) = v(s) + A(s,a)$.
18. **What does `.detach()` do in PyTorch?** Removes a tensor from the computation graph, stopping gradient flow.
19. **`nnet.train()` vs `nnet.eval()`?** Train mode enables dropout and batch norm updates. Eval mode disables them for deterministic inference.
20. **Why `loss.item()` not `loss`?** Using `loss` keeps the computation graph in memory, causing a memory leak.
21. **Universal approximation theorem?** A neural network with enough hidden units can approximate any continuous function. Does NOT guarantee we can find the right weights via training.
22. **SGD vs full gradient descent?** SGD uses random mini-batches instead of full dataset. Faster per

step, noisier gradients, but often converges well.

23. **What is $n$-step TD?** Intermediate between TD(0) and MC. Uses $n$ actual rewards then bootstraps. $n = 1$ is TD(0), $n = \infty$ is MC.

24. **DQN ablation key finding?** Both replay buffer AND target network are essential. Removing either one drastically hurts performance.

25. **Robbins-Monro conditions?** For convergence: $\sum \alpha_n = \infty$ (steps large enough to overcome initial conditions) and $\sum \alpha_n^2 < \infty$ (steps shrink enough to converge).